Project 1 Evaluation Details

Experimental Platform

Programming Language

Python Version: 3.10

Lib Version

`pymoo == 0.6.0.1`\
`pandas == 2.0.3`\
`numpy == 1.24.4`\
`scipy == 1.14.1`\
`networkx == 2.8.8`

Part 1 Evaluator

Input and Output Format of Algorithm

The executable solver of evaluator be named by `Evaluator.py`.

Input

1. The format of the evaluator call should be as follows:

```
python Evaluator.py –n <social network> -i <initial seed set> -b <balanced seed set> -k
<budget> -o <object value output path>
```

<social network> is the absolute path of the social network file, which is required to conform to the format of the social network specified below.\
<initial seed set> is the absolute path of the two campaigns' initial seed set, which is required to conform to the format of the seed set specified below.\
<balanced seed set> is the absolute path of the two campaigns' balanced seed set, which is required to conform to the format of the seed set specified below. Note that this is the input file path of the `Evaluator.py`.\
<budget> is a positive integer budget.\
<object value output path> is the output absolute path of the objective value.

Output

1. The output of the evaluator call should be the objective value written into the **<object value output path>**

```
234.43
```

The format of the social Network is as Follows:

1. The first line contains the number of nodes n and edges m
2. The following m line is the edge of the social network:\
   First column: ID of the source node, $u \in [0, n-1]$;\
   Second column: ID of the target node, $v \in [0, n-1]$;\
   Third column: the first campaign weight $p*(u,v)^1$ of edge;\
   Fourth column: the second campaign weight $p*(u,v)^2$ of edge;

The format of the seed set (initial seed set and balanced seed set) is as follows:

1. The first line contains the number of two seed sets k_1 and k_2;
2. The following k_1 line is the seed of campaign c_1;
3. The following k_2 line is the seed of campaign c_2;

Dataset

Table 1 provides an overall description of the test datasets:

1. **Type Column**: graph type, all datasets are directed graphs.
2. **Nodes Column**: number of nodes in the graph.
3. **Edges Column**: number of edges in the graph.
4. **TL Column**: Time limitation for tasks.

For **stage 1**, there are **2 test cases** in total:

Dataset	Type	Nodes	Edges	Usability TL	Baseline TL	Higher TL
Dataset 1	Directed	475	13,289	60s	60s	48s
Dataset 2	Directed	36,742	49,248	60s	60s	48s

Code Evaluation

In the programming aspects, there is **only one component in Phase 1**: the objective evaluation (2 points). After you submit your project, we will run some scripts to test your IEM solvers on different IEM problem instances. You will get the corresponding score once your solvers have passed a test.

The scoring rule for the objective evaluation is as follows. We will use two instances to test your evaluator, with each instance worth 1 point.

1. **Usability Test (0.2 points)**: In this test, we will check whether your estimator meets the input and output requirements. Please keep in mind that only those estimators that have passed test **1** will be evaluated in test **2**.
2. **Accuracy Test (0.5 points)**: In this test, we will assess the estimation error of your estimator. To be specific, we will establish a common cutoff time and an acceptable range for estimation values. We will then evaluate whether your estimated values fall within the specified range. Please keep in mind that only estimators that have successfully completed test **2** will undergo evaluation in test **3**.
3. **Efficiency Test (0.3 points)**: In this test, we will focus on how fast your estimator is. To be specific, we will set a common cutoff time and measure the running time of a baseline. Subsequently, we will

assess whether your estimator can complete its task in less time compared to the baseline.

In summary, the total score of this project is $2 = 2 \times (0.2 + 0.5 + 0.3)$.

Part 2 Heuristic

Input and Output Format of Algorithm

The executable solver of the algorithm must be named by `IEMP_Heur.py`.

Input

1. The format of the solver call should be as follows:

```
python IEMP_Heur.py –n <social network> -i <initial seed set> -b <balanced seed set> -k
<budget>
```

<social network> is the absolute path of the social network file, which is required to conform to the format of the social network specified below.\
<initial seed set> is the absolute path of the two campaigns' initial seed set, which is required to conform to the format of the seed set specified below.\
<balanced seed set> is the absolute path of the two campaigns' balanced seed set, which is required to conform to the format of the seed set specified below. Note that this is the output file path of the `IEMP_Heur.py`.\
<budget> is a positive integer budget.\
<object value output path> is the output absolute path of the objective value.

Output

1. The output of the solver call should be the balanced seed set of the two campaigns found by your algorithm written into the **<balanced seed set>**.

The format of the social Network is as Follows:

1. The first line contains the number of nodes n and edges m
2. The following m line is the edge of the social network:\
   First column: ID of the source node, $u \in [0, n-1]$;\
   Second column: ID of the target node, $v \in [0, n-1]$;\
   Third column: the first campaign weight $p * (u, v)^1$ of edge;\
   Fourth column: the second campaign weight $p * (u, v)^2$ of edge;

The format of the seed set (initial seed set and balanced seed set) is as follows:

1. The first line contains the number of two seed sets k_1 and k_2;
2. The following k_1 line is the seed of campaign c_1;
3. The following k_2 line is the seed of campaign c_2;

Grading Rules

Code Evaluation

In the programming aspects, there are **only one components in Phase 2**: the heuristic algorithm (6.5 points). After you submit your project, we will run some scripts to test your IEM solvers on different IEM problem instances. You will get the corresponding score once your solvers have passed a test.

The score rules for the heuristic algorithm are consistent and outlined as follows, with each algorithm worth 6.5 points.

1. **Efficacy Test (3.9 points)**: In this test, we will focus on how good your solvers are on **three** instances, each worth **1.3 points**.\
   Specifically, we will first check whether your solvers can satisfy the input and the output requirements (**test a, 0.1 points**). Subsequently, we will establish a common cutoff time and the solution quality of a baseline. We will then evaluate whether your solver can get a larger value of balanced information exposure within the given time (**test b, 0.9 point**). If your solver can get significantly better solution quality within the given time or complete the task in significantly less time compared to the baseline, you can get the bonus (**test c, 0.3 points**).\
   Please keep in mind that only solvers that have successfully completed test a will undergo evaluation in test b, and only solvers that have successfully completed test b will undergo evaluation in test c.
2. **Robustness Test (2.6 points)**: A robust solver is said to be applicable to a wide range of problem instances. Although your solvers may show good performance in the efficacy test, they may perform badly on unseen instances. In this test, for each participant solver, we will generate **two** new instances, each worth **1.3 points**.\
   On each instance, like the efficacy test, we will set a common cut-off time for all participant solvers and the balanced information exposure of a baseline. Subsequently, we will first check whether your solvers can satisfy the input and the output requirements (**test a, 0.1 points**) and evaluate whether your solver can get a larger value of balanced information exposure within the given time (**test b, 0.9 points**). If your solver can get significantly better solution quality within the given time or complete the task in significantly less time compared to the baseline, you can get the bonus (**test c, 0.3 points**).\
   *Please keep in mind that only solvers that have successfully completed test **a** will undergo evaluation in test **b**, and only solvers that have successfully completed test **b** will undergo evaluation in test **c**.*

In summary, the total score of this phase is $6.5 = 3 \times (0.1 + 0.9 + 0.3) + 2 \times (0.1 + 0.9 + 0.3)$.

Eval

For **stage 2**, there are **5 test cases** in total:

Case No.	Nodes	Edges	Baseline TL	Higher TL
case 0	475	13289	90s	30s
case 1	36742	49248	840s	540s
case 2	36742	49248	840s	540s
case 3	7115	103689	660s	450s
case 4	3454	32140	540s	420s

Note: Cases 0 ~ 2 have been released to you.

Detailed Requirements for Case 0 ~ Case 3

Case No.	Baseline	Higher Requirement	Baseline TL	Higher TL	K
case 0	430	450	90s	30s	10
case 1	35900	36035	840s	540s	15
case 2	36000	36200	840s	540s	15

TL: Short for Time Limits.

Grading Criteria

Take **case 0** for example:

- Initial point = 0.
- First, check if the solution satisfies the **input and output requirements**:\
  - If yes, add **0.1** to the point.\
    - If no, return **0**.
- Evaluate your output (the balanced seed sets) and get the objective value:\
  - If the objective value **>= 430 (Baseline)** AND the running time **<= 90s (Baseline TL)**, add **0.9** to the point.\
    - If the objective value **>= 450 (Higher requirement)** OR the running time **<= 30s (Higher TL)**, add **0.3** to the point.

Part 3 EA & Simulated Annealing

Input and Output Format of Algorithm

The executable solver of two kinds of algorithms must be named by `IEMP_Evol.py`.

Input

1. The format of the solver call should be as follows:

```
python IEMP_Evol.py –n <social network> -i <initial seed set> -b <balanced seed set> -k
<budget>
```

<social network> is the absolute path of the social network file, which is required to conform to the format of the social network specified below.\
<initial seed set> is the absolute path of the two campaigns' initial seed set, which is required to conform to the format of the seed set specified below.\

<balanced seed set> is the absolute path of the two campaigns' balanced seed set, which is required to conform to the format of the seed set specified below. Note that this is the output file path of the `IEMP_Evol.py`.\
<budget> is a positive integer budget.\
<object value output path> is the output absolute path of the objective value.

Output

1. The output of the solver call should be the balanced seed set of the two campaigns found by your algorithm written into the **<balanced seed set>**.

The format of the social Network is as Follows:

1. The first line contains the number of nodes n and edges m
2. The following m line is the edge of the social network:\
   First column: ID of the source node, $u \in [0, n-1]$;\
   Second column: ID of the target node, $v \in [0, n-1]$;\
   Third column: the first campaign weight $p * (u, v)^1$ of edge;\
   Fourth column: the second campaign weight $p * (u, v)^2$ of edge;

The format of the seed set (initial seed set and balanced seed set) is as follows:

1. The first line contains the number of two seed sets k_1 and k_2;
2. The following k_1 line is the seed of campaign c_1;
3. The following k_2 line is the seed of campaign c_2;

Grading Rules

Code Evaluation

In the programming aspects, there are **only one components in Phase 3**: the evolutionary algorithm (or simulated annealing algorithm)(6.5 points). After you submit your project, we will run some scripts to test your IEM solvers on different IEM problem instances. You will get the corresponding score once your solvers have passed a test.

The score rules for the evolutionary algorithm (or simulated annealing algorithm) are consistent and outlined as follows, with each algorithm worth 6.5 points.

1. **Efficacy Test (3.9 points)**: In this test, we will focus on how good your solvers are on **three** instances, each worth **1.3 points**.\
   Specifically, we will first check whether your solvers can satisfy the input and the output requirements (**test a, 0.1 points**). Subsequently, we will establish a common cutoff time and the solution quality of a baseline. We will then evaluate whether your solver can get a larger value of balanced information exposure within the given time (**test b, 0.9 point**). If your solver can get significantly better solution quality within the given time or complete the task in significantly less time compared to the baseline, you can get the bonus (**test c, 0.3 points**).\
   Please keep in mind that only solvers that have successfully completed test a will undergo evaluation in test b, and only solvers that have successfully completed test b will undergo evaluation in test c.
2. **Robustness Test (2.6 points)**: A robust solver is said to be applicable to a wide range of problem instances. Although your solvers may show good performance in the efficacy test, they may perform

badly on unseen instances. In this test, for each participant solver, we will generate **two** new instances, each worth **1.3 points**.\
On each instance, like the efficacy test, we will set a common cut-off time for all participant solvers and the balanced information exposure of a baseline. Subsequently, we will first check whether your solvers can satisfy the input and the output requirements (**test a, 0.1 points**) and evaluate whether your solver can get a larger value of balanced information exposure within the given time (**test b, 0.9 points**). If your solver can get significantly better solution quality within the given time or complete the task in significantly less time compared to the baseline, you can get the bonus (**test c, 0.3 points**).\
*Please keep in mind that only solvers that have successfully completed test **a** will undergo evaluation in test **b**, and only solvers that have successfully completed test **b** will undergo evaluation in test **c**.*

In summary, the total score of this phase is $6.5 = 3 \times (0.1 + 0.9 + 0.3) + 2 \times (0.1 + 0.9 + 0.3)$.

Eval

For **stage 3**, there are **5 test cases** in total：

Test Case Overview

Case No.	Nodes	Edges	Baseline TL	Higher TL
case 0	475	13289	420s	380s
case 1	13984	17319	860s	780s
case 2	13984	17319	860s	780s
case 3	3454	32140	1350s	1250s
case 4	3454	32140	1350s	1250s

Note: Cases 0 ~ 2 have been released to you.

Detailed Requirements for Case 0 ~ Case 2

Case No.	Baseline	Higher Requirement	Baseline TL	Higher TL	K
case 0	415	440	420s	380s	10
case 1	13580	13680	860s	780s	14
case 2	13350	13600	860s	780s	14

TL: Short for Time Limits.

Grading Criteria

Take **case 0** for example:

- Initial point = 0.
- First, check if the solution satisfies the **input and output requirements**:

    - If yes, add **0.1** to the point.
    - If no, return **0**.
- Evaluate your output (the balanced seed sets) and get the objective value:
    - If the objective value **>= 415 (Baseline)** AND the running time **<= 420 (Baseline TL)**, add **0.9** to the point.
    - If the objective value **>= 440 (Higher requirement)** OR the running time **<= 380 (Higher TL)**, add **0.3** to the point.

Submission Details

All files are required to submit via blackboard system in a compressed file named exactly <Project1_[Student ID]>, which contains:

- report.pdf
- Evaluator.py
- IEMP_Heur.py
- IEMP_Evol.py

An incorrect number of files may cause automatic evaluation to fail.

Each student is allowed **only one** opportunity to test their code before the final submission. Please send your code to jianghl2025@mail.sustech.edu.cn with the subject line: **"Project 1 Testing - [Student ID]"**. A feedback report will be provided upon processing.

Report Example:

```
[Heuristic / map1]  得分: 1.3
  [Usability] ✓ 结果=436.99, 得 0.1 分
  [Accuracy] ✓ 结果=436.99, 得 0.9 分
  [Efficiency] ✓ Accuracy时间内结果已达标 (=436.99), 得 0.3 分

[Heuristic / map2]  得分: 0.1
  [Usability] ✓ 结果=35689.44, 得 0.1 分
  [Accuracy] ✗ 结果=35689.44, 不在范围 [35900, 36742], 得 0 分
  [Efficiency] ✗ 结果不够好 (Accuracy时=35689.44, Efficiency时=35691.52), 得 0 分
....

▸ Evaluator
    map1    →  1.0 分
    map2    →  1.0 分
  Evaluator 小计: 2.0 分

▸ Heuristic
    map1    →  1.3 分
    map2    →  0.1 分
    map3    →  0.1 分
    map4    →  0.1 分
```

```
    map5    →  1.3 分
  Heuristic 小计: 2.9 分

▸ Evolutionary
    map1    →  1.3 分
    map2    →  1.3 分
    map3    →  1.3 分
    map4    →  1.3 分
    map5    →  1.3 分
  Evolutionary 小计: 6.5 分

────────────────────────────────────────────────────────────────────────────────
  总分: 11.4
```

Reference

[1] K Garimella, A Gionis, N Parotsidis, N Tatti. Balancing information exposure in social networks. NeurIPS 2017: 4663-4671

[2] S Cheng, H Shen, J Huang, W Chen, X Cheng. IMRank: influence maximization via finding self-consistent ranking. SIGIR 2014: 475-484

[3] M Gong, J Yan, B Shen, L Ma, Q Cai. Influence maximization in social networks based on discrete particle swarm optimization. Inf. Sci. 367-368: 600-614 (2016)

[4] Q Jiang, G Song, G Cong, Y Wang, W Si, K Xie. Simulated Annealing Based Influence Maximization in Social Networks. AAAI 2011